

IMLO Practical Labs - Semester 2 / Week 7

1 Log loss derivative

In the lecture, we looked at how to optimise objective functions. One way of doing that is by applying *gradient descent*. For that, we need to compute the derivative of an objective function. In this exercise, you'll work through the derivative of the negative log likelihood function.

(a) Work through either of the two blog posts linked below on your own, and derive the derivative of the negative log likelihood loss function w.r.t. θ by yourself. Recall: $\mathcal{L}_{NLL}(g, y) = -(y \log(g) + (1 - y) \log(1 - g))$.

- Option 1: *The Derivative of Cost Function for Logistic Regression* by Saket Thavanani <https://medium.com/analytics-vidhya/derivative-of-log-loss-function-for-logistic-regression-9b832f025c2d>
- Option 2: Section *The cross-entropy cost function* (and following), *Neural Networks and Deep Learning* book by Michael Nielsen http://neuralnetworksanddeeplearning.com/chap3.html#the_cross-entropy_cost_function. It is very nicely explained here but we haven't covered everything that they mention yet. What they call "weights" are our (θ, θ_0) , and a "neuron" is basically simply a linear classifier with sigmoid function.

I am linking two options here, as different people might find one or the other more clear - and it is also often helpful to read the same thing explained by two different people, to understand it. The whole book from Michael Nielsen is really good in particular, and we'll cover some of the neural networks things that he mentions in the next weeks.

2 Gradient Descent

(a) Given the function $f(x) = (3x + 4)^2$. Write down the update step for a gradient descent iteration on paper. More concretely: Given an initial guess $x^{(0)}$, write down the update step to compute $x^{(1)}$.

(b) What is the optimal value of x that minimises f ? (Calculate on paper.)

(c) Define your function from (a) in Python as follows, near the top of the provided skeleton file `week_7_8.py`:

```
def f1(x):  
    return float((3 * x + 4)**2)
```

And define a function that returns its gradient (derivative):

```
def df1(x):  
    return ... # You will have computed this in (a)
```

We will use this function and its derivative in a moment, when implementing our own version of gradient descent.

(d) We now want to implement a generic version of gradient descent that can take any arbitrary scalar Python function, like `f1(x)` from (c), and its gradient `df`. Use the provided skeleton file `week_8_9.py`, where you can find a function definition with an empty body:

```
def gradient_descent(f, df, x0, step_size, max_iter):  
    ''' ...  
    '''  
    pass
```

Make sure to read the docstring of that function, to understand what input and output parameters the function should have. In the (empty) function body, implement the gradient descent algorithm.

Remark: We could use either a termination criterion for the algorithm (like in the lecture slides), or just run the algorithm for a given number of `max_iter`. You can use either, but for simplicity and testing purposes, I suggest using `max_iter`.

(e) We'll try the `gradient_descent` function with the earlier example function.

- i) First, plot the function from (c) using matplotlib. Hint: You'll have to use `np.linspace(...)` for this, evaluate the function for a given interval, and then plot it.
- ii) Plot the initial value `x0` into the same figure.

- iii) Call the `gradient_descent` function with the functions `f1` and `df1` that you've written in (c). Experiment with different starting values, step sizes (try step sizes between 1 and 0.0001), and number of iterations. If your implementation doesn't converge for any step size, the next steps might help you to debug what might be going wrong. If your implementation is correct, then for a well-chosen step size, your algorithm should find the same minimum that you've calculated by hand in (b).
 - iv) We will visualise what gradient descent is doing in each step. Your `gradient_descent` function should return the x value of the final step, and a list of all x values xs and $f(x)$ values fs , one each per iteration. Plot those values into your figure where you've plotted the function, and observe the behaviour of gradient descent.
 - v) Run your implementation of gradient descent with a variety of step sizes and initial values. When the optimisation converge? What step size is good, and what happens if the step size is either way too big or way too small?
- (f) Is the function $f(x)$ from (a) convex, and what does that mean for the convergence of gradient descent?